

E-STORE

Frontend Angular 17

Composants standalone, services réactifs, interceptors

Module	Full-Stack
Encadrant	Pr. Omar Zahour
Établissement	FSBM — Université Hassan II
Année	2025-2026
Auteurs	Akram Belmoussa & Nouhaila Ben Soumane
Document	4/7 d'une série explicative

1. Choix techniques Angular 17

- **Standalone components** — pas de NgModule, déclarations dans la classe.
- **Lazy loading** par route — chaque feature est chargée à la demande (bundles plus petits).
- **Signals** pour l'état réactif (panier, toasts) — alternative moderne à RxJS pour l'UI.
- **HttpInterceptor fonctionnel** (Angular 15+) — code plus concis qu'une classe.
- **Reactive Forms** avec validation déclarative.
- **Bootstrap 5** pour le style — pas de dépendance JS, juste le CSS.

2. Structure du dossier src/app

src/app/	
■■■ app.config.ts	Configuration globale (router, http, interceptors)
■■■ app.routes.ts	Définition des routes avec lazy-loading
■■■ app.component.ts	Layout racine (header + outlet + footer)
■■■ core/	
■ ■■■ models/	Interfaces TypeScript (User, Product, Cart...)
■ ■■■ services/	Services HTTP (AuthService, CartService...)
■ ■■■ guards/	AuthGuard
■ ■■■ interceptors/	AuthInterceptor + ErrorInterceptor
■■■ shared/	
■ ■■■ components/	HeaderComponent, FooterComponent, LoaderComponent, ToastComponent
■■■ features/	
■■■ auth/	login.component.ts, register.component.ts
■■■ catalog/	product-list, product-detail
■■■ cart/	cart.component.ts
■■■ orders/	orders.component.ts
■■■ profile/	profile.component.ts
■■■ reviews/	review-form.component.ts

3. AuthService — gestion de l'utilisateur courant

```

@Injectable({ providedIn: 'root' })
export class AuthService {
  private http = inject(HttpClient);
  private currentUser$ = new BehaviorSubject<User | null>(this.loadUser());
  readonly user$ = this.currentUser$.asObservable();

  login(email: string, password: string) {
    return this.http
      .post<ApiResponse>(<AuthResponse>&gt;(`${env.apiUrl}/auth/login`, { email, password }))
      .pipe(tap((res) => res.data & & this.persist(res.data)));
  }

  private persist(data: AuthResponse): void {
    localStorage.setItem('estore.token', data.token);
    localStorage.setItem('estore.user', JSON.stringify(data.user));
    this.currentUser$.next(data.user);
  }
}

```

Le token et l'utilisateur sont stockés en **localStorage** pour survivre au rafraîchissement. Un **BehaviorSubject** permet aux composants de réagir aux changements.

4. CartService avec signals

```
@Injectable({ providedIn: 'root' })
export class CartService {
  private http = inject(HttpClient);
  readonly cart = signal<Cart | null>(null);
  readonly itemCount = signal<number>(0);

  add(productId: number, quantity: number) {
    return this.http
      .post<ApiResponse>(Cart&&(`${env.apiUrl}/cart/add`, { productId, quantity }))
      .pipe(tap((r) => this.update(r.data)));
  }

  private update(cart: Cart | undefined): void {
    if (!cart) return;
    this.cart.set(cart);
    this.itemCount.set(cart.itemCount);
  }
}
```

Le badge du panier dans le header utilise directement `cartSvc.itemCount()` — Angular détecte automatiquement la dépendance et met à jour le DOM.

5. ErrorInterceptor — gestion centralisée

```
export const errorInterceptor: HttpInterceptorFn = (req, next) => {
  const toast = inject(ToastService);
  const auth = inject(AuthService);

  return next(req).pipe(
    catchError((err: HttpErrorResponse) => {
      const msg = err.error?.message || `Erreur HTTP ${err.status}`;
      if (err.status === 401) {
        auth.logout();
        toast.error('Session expirée, veuillez vous reconnecter');
      } else if (err.status !== 0) {
        toast.error(msg);
      } else {
        toast.error('Serveur injoignable');
      }
      return throwError(() => err);
    })
  );
};
```

Aucun composant n'a besoin de gérer manuellement les erreurs HTTP : un toast approprié s'affiche automatiquement, et un 401 déclenche la déconnexion + redirection vers `/login`.

6. Routes paresseuses (lazy loading)

```
export const routes: Routes = [
  { path: '', loadComponent: () =>
    import('./features/catalog/product-list.component').then((m) => m.ProductListComponent) },
  { path: 'cart',
    loadComponent: () => import('./features/cart/cart.component').then((m) => m.CartComponent),
    canActivate: [authGuard]
  },
  { path: 'orders',
    loadComponent: () => import('./features/orders/orders.component').then((m) => m.OrdersComponent)
  }
];
```

```
    canActivate: [authGuard]
  },
  // ...
];
```

Chaque route charge son composant à la demande — le bundle initial reste léger (13 ko de main.js + 245 ko de framework partagé), les features sont des bundles indépendants chargés au besoin.